Proyecto Dron Autonomo

Contexto tecnico y enfoques de trabajo | Grupo estudiantil de drones | TMR 2026

Este documento resume el estado del proyecto, las decisiones tomadas, el entorno de desarrollo ya instalado y los enfoques propuestos. Esta escrito para servir como contexto inicial en una sesion de trabajo con codigo, de modo que no haya que reexplicar el proyecto.

1. Objetivo del proyecto

Construir y programar un dron autonomo en el marco de un grupo estudiantil universitario, con dos objetivos paralelos: (a) aprender y dominar el ecosistema ArduPilot, y (b) competir en la categoria de Drones Autonomos del Torneo Mexicano de Robotica 2026, cuyas misiones se basan en la categoria de interiores del IMAV 2025.

El interes principal esta en la Mision 3: localizar una caja que contiene una bocina emitiendo un sonido de auxilio y entregarle un med-kit.

2. Hardware

2.1 Dron en construccion

Componente	Detalle y notas relevantes
Controladora de vuelo	Holybro Pixhawk 6C Mini (FMUv6C, STM32H743). Firmware en Mission Planner: Pixhawk6C. Para DShot b
ESC	Tekko32 con firmware AM32. Protocolo DShot: no requiere calibracion de ESC. Configuracion via pass-thro
Computadora a bordo	Raspberry Pi 5. Requiere BEC dedicado de 5V / 5-6A (NO alimentar del riel de 5V de la controladora). Enlace prefer
Frame	Fibra de carbono (material conductor). Hay continuidad entre tornillos: es normal. Verificar antes de conectar bateria

2.2 Hardware de practica y candidatos

- **Pixhawk 2.4.8** (se identifica como fmuv3, reconocida con 2 MB de flash, es decir sin recorte de funciones). Ya volo previamente con control manual. Se usa para practicar conexion, calibraciones, lectura de parametros y analisis de logs reales.
- **Hexacoptero ofrecido**: pendiente de evaluar. Ventajas: carga util y tolerancia a falla de un motor. Riesgo: el reglamento del TMR limita a 60 cm de punta de helice a punta opuesta, y un hexa tipico no cabe. **Medir antes de comprometerse.**

2.3 Parametros clave de ESC y filtrado (para cuando se configure)

```
MOT_PWM_TYPE = 6 # DShot600
SERVO_BLH_AUTO = 1 # pass-through para configurar ESC
SERVO_BLH_BDMASK # 15 (4 canales sin IOMCU) / 3840 (4 canales AUX con IOMCU)
SERVO_BLH_POLES = 14 # polos del motor (valor tipico)
INS_HNTCH_ENABLE = 1 # notch armonico
INS_HNTCH_MODE = 3 # fuente de RPM: telemetria de ESC
INS_HNTCH_REF = 1
INS_HNTCH_FREQ / INS_HNTCH_BW # segun frecuencia de vuelo estacionario
```

Estos valores se aplican despues del primer vuelo, no antes. Confirmar siempre contra la wiki.

2.4 Aislamiento electrico y montaje (frame de carbono)

- Separadores (standoffs) y tornilleria de nylon como metodo principal. La cinta aislante no es una solucion de montaje: se afloja con calor y vibracion.
- Bases impresas en 3D (PETG o ABS, no PLA) para Raspberry Pi y perifericos.
- Lamina de G10/FR4 o cinta Kapton para aislamiento puntual entre placa y carbono.
- Pasacables de goma o termocontraible en todo cruce de cable por borde de carbono.
- El carbono atenua RF: antenas de GPS, telemetria y RC deben sobresalir de la estructura.
- No aplicar barniz conformal a la controladora (cubre el barometro).

3. Entorno de desarrollo instalado

3.1 Ubuntu 24.04 (particion principal de trabajo)

- Usuario agregado al grupo **dialout**; ModemManager enmascarado (interferia con puertos serie).
- **QGroundControl** como AppImage en el directorio personal (estacion de tierra nativa en Linux).
- **ArduPilot** clonado en ~/ardupilot con submodulos; script install-prereqs-ubuntu.sh ejecutado correctamente. Entorno virtual de Python en ~/venv-ardupilot con MAVProxy, pymavlink, wxPython y OpenCV. Toolchain ARM (gcc-arm-none-eabi) instalado.
- **SITL** funcionando: se valido un vuelo simulado completo (Guided, arm, takeoff, RTL).
- **ROS 2 Jazzy y Gazebo** ya estaban instalados en el sistema (visible en el PATH).

```
# Simulador
cd ~/ardupilot/ArduCopter
sim_vehicle.py --console --map

# Simulador con salida extra para scripts propios (14550 queda libre para QGC)
sim_vehicle.py --console --map --out=udp:127.0.0.1:14551

# Cargar parametros de un dron real dentro del simulador (desde MAVProxy)
param load ~/ruta/parametros.parm

# Estacion de tierra
~/QGroundControl-x86_64.AppImage &
```

3.2 Windows (particion secundaria)

Solo **Mission Planner** (instalador .msi, incluye drivers). Es la herramienta que usa el resto del club, por lo que es la referencia comun del equipo. No requiere Python ni dependencias extra: su simulador descarga binarios de SITL ya compilados. Mission Planner se usara para instalacion de firmware, calibraciones, tuning fino y analisis de logs.

3.3 Division de herramientas acordada

Tarea	Herramienta
Configuracion, firmware, tuning, analisis de logs	Mission Planner (Windows)
Vuelo y configuracion basica desde Linux	QGroundControl (Ubuntu)

Simulacion, desarrollo, scripts, ROS 2	SITL + MAVProxy + Python (Ubuntu)
Analisis de logs en navegador	UAV Log Viewer (plot.ardupilot.org)

Se descarto usar ArduPilot Methodic Configurator como herramienta, por decision del club. Su guia de tuning se conserva unicamente como lista de lectura del orden de configuracion.

4. Conceptos base de ArduPilot (resumen de referencia)

- **Capas**: ArduPilot es el firmware dentro de la controladora; Mission Planner, QGroundControl y MAVProxy son estaciones de tierra; MAVLink es el protocolo entre ambos; el control remoto es el enlace de seguridad del piloto.
- **Parametros**: toda la configuracion vive en parametros. Las interfaces graficas solo los modifican. Respaldar el archivo completo tras cada etapa importante.
- **Modos de vuelo (Copter)**: Stabilize, AltHold, Loiter/PosHold, Guided (ordenes desde codigo), Auto (mision cargada), RTL, Land.
- **Armado**: bloqueado por pre-arm checks; los mensajes explican el motivo del rechazo. No desactivarlos.
- **Failsafes**: radio, bateria, telemetria y EKF. Definen el comportamiento ante fallas.
- **EKF**: fusiona sensores para estimar posicion y actitud. Sus errores suelen venir de vibraciones, brujula mal calibrada o interferencia magnetica.
- **Logs**: archivos .bin en la SD de la controladora. Campos utiles: VIBE, ATT, RCOU, BAT, GPS, MSG, ERR.

5. Reto TMR 2026 - restricciones que condicionan el diseno

Restriccion	Implicacion tecnica
Arena interior de 10 x 10 x 4 m con red	No hay GPS. Es el problema central: se requiere otra fuente de posicion (flujo optico + telemet
Maximo 60 cm de punta de helice a punta de hel	Empuja a un dron pequeno (clase 5-7 pulgadas). El peso de la Pi 5 con disipador es significativo a esa
Prohibido el pilotaje manual, incluidos despeg	Solo se permite accionar un boton o comando; la ejecucion debe ser autonoma. Encaja con control en
Mecanismo de paro o aterrizaje de emergencia	Sin el no se permite participar. Debe demostrarse al juez.
Hasta 5 cajas identicas (57 x 78 x 36 cm), solo	El sonido sirve para reconocer el sonido, sino para desempatar entre cajas detectadas visua
Bocina a minimo 70 dB medidos a 1 m, aportada	Conviene tenerla en laboratorio para las pruebas.
Med-kit de 20 x 10 x 5 cm y 30 g	Requiere mecanismo de suelta y precision de posicion del orden de +/- 20 cm.
Multiplicadores de puntaje	Procesamiento a bordo x1.5; enjambre con procesamiento a bordo x3; la presentacion ante jueces sur
Opcion de renunciar a la busqueda	Se puede pedir la ubicacion aproximada de la caja y aun asi puntuar por la entrega. Ruta segura para

6. Enfoques propuestos

6.1 Posicionamiento en interiores (prioridad maxima)

Sin esto, ninguna otra mision es posible. Dos caminos, no excluyentes: sensor de flujo optico con telemetro laser hacia abajo (camino mas probado y sencillo en ArduPilot), u odometria visual-inercial calculada en la Pi y

enviada al EKF3 por MAVLink como fuente externa de posicion. Todo debe validarse primero en SITL.

6.2 Control autonomo

Modo **Guided** comandado desde la Raspberry Pi con **pymavlink** o **MAVSDK** (DroneKit esta descontinuado, no usarlo). Alternativa a futuro: ROS 2 via MAVROS o el soporte nativo de DDS de ArduPilot, aprovechando que ROS 2 Jazzy ya esta instalado. Todo el desarrollo puede hacerse contra SITL sin hardware.

```
SERIAL2_PROTOCOL = 2 # MAVLink2 hacia la computadora a bordo
SERIAL2_BAUD = 921 # 921600 bps
FENCE_* # geocerca antes de cualquier vuelo autonomo
```

6.3 Deteccion acustica: busqueda diferencial

El enfoque propuesto evita el problema de estimar direccion del sonido en pleno vuelo, que es muy dificil con el ruido de los rotores:

- Detectar las cajas por vision (son objetos grandes y de color uniforme).
- Sobrevolar cada caja a la **misma altura y con el mismo regimen de motores**, midiendo durante unos segundos la energia acustica en la banda de voz (300-3000 Hz).
- La caja correcta sera la de mayor energia. Al mantener condiciones de vuelo identicas, el ruido propio se vuelve casi constante y se cancela en la comparacion: **no se necesita SNR positivo, solo diferencia medible entre posiciones.**
- Ventaja del hardware elegido: la telemetria de RPM por DShot bidireccional de los ESC AM32 permite conocer las frecuencias tonales de las helices y suprimirlas de forma dirigida.

6.4 Clasificacion con YAMNet

YAMNet (MobileNet v1, 521 clases de AudioSet) espera audio **mono a 16 kHz normalizado a [-1, 1]** y devuelve puntuaciones, embeddings y espectrograma. Clases utiles: habla, grito, llanto. El valor principal esta en los **embeddings**: permiten entrenar un clasificador propio con grabaciones reales del equipo (con ruido de rotor incluido) sin entrenar desde cero. Para la Pi conviene la version TFLite. Los enlaces antiguos a tfhub.dev pueden redirigir a Kaggle Models.

Plan: ejecutar en paralelo **YAMNet** y la **medicion de energia en banda** sobre las mismas grabaciones. Hipotesis de trabajo: YAMNet funcionara bien sin dron y se degradara con motores encendidos, mientras que la energia en banda seguira discriminando entre caja correcta y caja vacia.

6.5 Hardware de audio

Opcion	Uso y notas
Microfono USB o adaptador USB + solapa	Para empezar de inmediato en laptop o Pi. Consecucion local sencilla.
Arreglo de microfonos USB (tipo ReSpeaker)	Recomendado para el proyecto: clase audio USB sin drivers, beamforming y estimacion de direccion in
MEMS I2S (INMP441 / ICS-43434)	Mas ligero y digital desde el sensor, ideal para el dron final. En la Pi 5 la configuracion de I2S por GPIO
Capsula electret analogica	Descartada: la Raspberry Pi no tiene entrada analogica ni ADC.

6.6 Banco de pruebas de audio

- Bocina capaz de 70 dB a 1 m, apuntando hacia arriba, dentro de una caja de plastico de dimensiones similares a la del reglamento (la caja actua como caja acustica y altera el espectro).
- Sonometro (aplicacion de celular) para repetibilidad entre pruebas.
- Espuma antiviento en el microfono: obligatoria en presencia de helices.
- Audio de referencia grabado por el equipo, no descargado.
- Protocolo: grabar 10 s a 1, 2 y 3 m de altura, con y sin dron encendido; analizar espectrogramas.

```
arecord -l
arecord -D plughw:1,0 -f S16_LE -r 16000 -c 1 prueba.wav
```

7. Orden de trabajo propuesto

Etapa	Contenido
1	Dron volando y estable: firmware, calibraciones, DShot, primer vuelo, AutoTune, revision de vibraciones en logs.
2	Posicionamiento en interiores sin GPS. Validar en SITL y luego en vuelo real.
3	Enlace Raspberry Pi - controladora. Primero solo lectura de telemetria, sin comandos.
4	Control en Guided desde codigo, en area abierta y baja altura, con geocerca y piloto listo.
5	Vision: deteccion de cajas y del marcador ARUCO. Mecanismo de suelta del med-kit.
6	Audio: busqueda diferencial y clasificacion. Es la ultima capa; depende de todo lo anterior.

Regla general adoptada: cada linea de codigo que controle el dron se prueba primero en SITL. Siempre hay un piloto con el radio listo para retomar el control, incluso en vuelo autonomo.

8. Pendientes y preguntas abiertas

- Medir la diagonal del dron en construccion y del hexacoptero ofrecido (limite: 60 cm).
- Definir motores, helices y bateria, para dimensionar consumo y margen de carga.
- Confirmar con la organizacion del TMR si habra cambios en el reglamento 2026 respecto al documento basado en IMAV 2025.
- Consultar a la organizacion si se permite colocar microfonos en el piso de la arena. De permitirse, la localizacion podria hacerse antes del despegue, con el dron apagado y SNR excelente.
- Obtener logs recientes del hexacoptero candidato para evaluar su estado real.
- Verificar continuidad frame-positivo antes de la primera conexion de bateria.

9. Notas para la sesion de codigo

Stack disponible y rutas relevantes:

```
Ubuntu 24.04 | ROS 2 Jazzy | Gazebo
~/ardupilot # codigo fuente de ArduPilot
~/venv-ardupilot # entorno Python (MAVProxy, pymavlink, OpenCV)
~/QGroundControl-x86_64.AppImage
SITL: sim_vehicle.py -v ArduCopter --console --map --out=udp:127.0.0.1:14551
```

Tareas candidatas para abordar con codigo, en orden de utilidad inmediata:

- Script de conexion y lectura de telemetria con pymavlink contra SITL (heartbeat, modo, posicion).
- Script de mision minima en Guided: armar, despegar a altura fija, desplazarse a coordenadas locales, aterrizar. Con verificaciones de seguridad y timeouts.
- Herramienta de analisis de audio: grabacion, filtrado en banda 300-3000 Hz, calculo de energia por ventana y comparacion entre posiciones.
- Integracion de YAMNet (TFLite) sobre grabaciones propias y extraccion de embeddings para entrenar un clasificador ligero.
- Nodo o script que combine deteccion visual de cajas con la medicion acustica por posicion.

Documento de contexto generado a partir de la sesion de asesoria tecnica del proyecto. Los valores de parametros deben confirmarse siempre contra la documentacion oficial de ArduPilot, que es la unica fuente actualizada para el hardware descrito.